



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

ESCOM

“Prototipo de diseño de un ISA y procesador monociclo didáctico de 8 bits basado en RISC-V”

Manual del Programador

Autores

**Pablo Isaac Macias Huerta
Juan Rubén Pérez Bárcenas**

Directores

**M. en C. Víctor Hugo García Ortega
Ing. Adbel Anahí Montes Meza**

Junio 2026

Capítulo 1

Introducción

1.1. Formatos de Instrucción

Las instrucciones disponibles necesitan un formato específico para poder funcionar correctamente y evitar que se mezclen señales, dando resultados incorrectos. Por ello, las instrucciones se dividen en tipos, dependiendo sus necesidades (la cantidad de registros que necesitan, ver si necesitan valores inmediatos) y sus funciones.

Los formatos de instrucción de la plataforma se presentan en la Fig. 1.1.

15	14	13	11	10	8	7	6	4	3	0	
funct2		rs2		rs1		funct1	rd		Opcode		R
imm[15:11]				rs1		funct1	rd		Opcode		I
imm[15:14]		rs2		rs1		funct1	imm[6:4]		Opcode		B
imm[15:11]				rs1		funct1	imm[6:4]		Opcode		S
imm[15:8]						funct1	rd		Opcode		J / U

Figura 1.1: Formatos de Instrucción del procesador

Las Instrucciones R son aquellas que son operaciones aritméticas, lógicas, de bits o de comparación entre registros. Estas tienen su respectivo símil que utiliza valores inmediatos (es decir, valores codificados dentro de la misma instrucción), que corresponden a las Instrucciones I.

Las Instrucciones B son para “Branch”, o saltos condicionales. Las S son para guardado de datos, las J para saltos incondicionales; y las U para carga de datos.

“Opcode” se refiere al código de operación de la instrucción, que es un identificador de 1 (o un grupo) de instrucciones. Para poder definir con precisión cuál instrucción es, se utilizan los códigos de función (funct1 y funct2). Todos estos elementos permiten la identificación de 1 de las 20 instrucciones disponibles.

Las celdas marcadas con r (rs1, rs2, rd) corresponden a un código de 3 bits que direcciona a 1 de 8 registros disponibles. rd se refiere al registro destino (donde se guardará el resultado), mientras que rs1 y rs2 son registros fuente (source), que se tomarán como operandos para la instrucción. Es importante recalcar que el registro x0 (con código 000) no es modificable y siempre tiene un valor de 0. Cada registro puede albergar un valor de hasta 8 bits.

Por último, los valores marcados con imm son valores inmediatos. Estos no necesariamente se guardan de manera contigua, y pueden ser signados o no, dependiendo de la instrucción.

1.2. Instrucciones disponibles

Las instrucciones disponibles para el procesador, así como su formato específico, se pueden encontrar en la Fig. 1.2. Los valores inmediatos marcados con una s (s_imm) son valores signados, mientras que los marcados con u (u_imm) son valores sin signar. Es importante tener en mente los rangos de valores disponibles dependiendo la longitud en bits.

15	14	13	11	10	8	7	6	4	3	0		
u_imm[15:8]					1	rd	0110		J	jal		
s_imm[15:14]	rs2		rs1		0	s_imm[6:4]		0110		B	beq	
s_imm[15:8]					0	rd	0001		U	li		
u_imm[15:8]					1	rd	0001		U	ld		
u_imm[15:11]			rs1		0	u_imm[6:4]		0010		S	st	
s_imm[15:11]			rs1		0	rd	1000		I	jalr		
s_imm[15:11]			rs1		0	rd	0011		I	addi		
s_imm[15:11]			rs1		1	rd	0011		I	subi		
s_imm[15:11]			rs1		0	rd	0100		I	xori		
s_imm[15:11]			rs1		1	rd	0100		I	andi		
s_imm[15:11]			rs1		0	rd	0111		I	slti		
u_imm[15:11]			rs1		0	rd	0101		I	slli		
u_imm[15:11]			rs1		1	rd	0101		I	srl		
00	rs2		rs1		0	rd	0000		R	add		
00	rs2		rs1		1	rd	0000		R	sub		
01	rs2		rs1		0	rd	0000		R	xor		
01	rs2		rs1		1	rd	0000		R	and		
10	rs2		rs1		0	rd	0000		R	slt		
10	rs2		rs1		1	rd	0000		R	sll		
11	rs2		rs1		0	rd	0000		R	srl		

Figura 1.2: Conjunto de Instrucciones del procesador propuesto

1.3. Instrucciones desarrolladas

En la Tabla 1.1 se pueden ver las instrucciones desarrolladas con una explicación corta y su formato específico, incluyendo códigos de operación (opcode) y códigos de función (funct1, funct2). Esta tabla se recomienda como consulta rápida, pero una explicación detallada de cada instrucción se puede encontrar en el Capítulo 2.

Instrucción	Estructura	Significado	funct2			funct1		Opcode	Tipo
Instrucciones de Carga y Almacenamiento									
LI	LI rd, immediate	x[rd] = immediate	immediate (8)			0	rd (3)	0001	U
LD	LD rd, offset	x[rd] = M[offset]	offset (8)			1	rd (3)	0001	U
ST	ST rs1, offset	M[offset] = x[rs1]	offset (8-4)		rs1 (3)	0	offset (3-1)	0010	S
Operaciones Aritméticas									
ADD	ADD rd, rs1, rs2	x[rd] = x[rs1] + x[rs2]	00	rs2 (3)	rs1 (3)	0	rd (3)	0000	R
ADDI	ADDI rd, rs1, immediate	x[rd] = x[rs1] + sext(immediate)	immediate (5)		rs1 (3)	0	rd (3)	0011	I
SUB	SUB rd, rs1, rs2	x[rd] = x[rs1] - x[rs2]	00	rs2 (3)	rs1 (3)	1	rd (3)	0000	R
SUBI	SUBI rd, rs1, immediate	x[rd] = x[rs1] - sext(immediate)	immediate (5)		rs1 (3)	1	rd (3)	0011	I
Operaciones Lógicas									
XOR	XOR rd, rs1, rs2	x[rd] = x[rs1] ⊕ x[rs2]	01	rs2 (3)	rs1 (3)	0	rd (3)	0000	R
XORI	XORI rd, rs1, immediate	x[rd] = x[rs1] ⊕ sext(immediate)	immediate (5)		rs1 (3)	0	rd (3)	0100	I
AND	AND rd, rs1, rs2	x[rd] = x[rs1] & x[rs2]	01	rs2 (3)	rs1 (3)	1	rd (3)	0000	R
ANDI	ANDI rd, rs1, immediate	x[rd] = x[rs1] & sext(immediate)	immediate (5)		rs1 (3)	1	rd (3)	0100	I
Comparadores									
SLT	SLT rd, rs1, rs2	x[rd] = (x[rs1] < x[rs2]) ? 1 : 0	10	rs2 (3)	rs1 (3)	0	rd (3)	0000	R
SLTI	SLTI rd, rs1, immediate	x[rd] = (x[rs1] < sext(immediate)) ? 1 : 0	immediate (5)		rs1 (3)	0	rd (3)	0111	I
Manipulación de Bits									
SLL	SLL rd, rs1, rs2	x[rd] = x[rs1] x[rs2]	10	rs2 (3)	rs1 (3)	1	rd (3)	0000	R
SLLI	SLL rd, rs1, immediate	x[rd] = x[rs1] sext(immediate)	immediate (5)		rs1 (3)	0	rd (3)	0101	I
SRL	SRL rd, rs1, rs2	x[rd] = x[rs1] x[rs2]	11	rs2 (3)	rs1 (3)	0	rd (3)	0000	R
SRLI	SRLI rd, rs1, immediate	x[rd] = x[rs1] sext(immediate)	immediate (5)		rs1 (3)	1	rd (3)	0101	I
Branch y Saltos									
BEQ*	BEQ rs1, rs2, offset	if(x[rs1] == x[rs2]) pc += sext(offset)	offset (5-4)	rs2 (3)	rs1 (3)	0	offset (3-1)	0110	B
JAL	JAL rd, offset	x[rd] = pc+1; pc = offset	immediate (8)			1	rd (3)	0110	J
JALR	JALR rd, rs1, offset	x[rd] = pc+1; pc = (x[rs1] + sext(offset))	immediate (5)		rs1 (3)	0	rd (3)	1000	J

Tabla 1.1: Formatos por Instrucción

Listado de instrucciones

add rd, rs1, rs2 x[rd] = x[rs1] + x[rs2]
Add. Tipo R.
 Suma el registro x[rs2] al registro x[rs1] y escribe el resultado en x[rd]. Overflow aritmético ignorado.

15	14	13	11	10	8	7	6	4	3	0
00		rs2		rs1		0	rd		0000	

15	11	10	8	7	6	4	3	0
immediate[15:11]		rs1		0	rd		0011	

15	14	13		11	10		8	7	6		4	3		0
01		rs2			rs1		1	rd		0000				

15	11	10	8	7	6	4	3	0
immediate[15:11]		rs1		1	rd		0100	

Si el registro $x[rs1]$ es igual al registro $x[rs2]$, asignar al PC su valor actual mas el offset extendido en signo.

sll rd, rs1, rs2 $x[rd] = x[rs1] \ll x[rs2]$
Shift Left Logical. Tipo R.

Corre el registro $x[rs1]$ a la izquierda por $x[rs2]$ posiciones de bits. Los bits liberados son reemplazados por ceros; y el valor es guardado en $x[rd]$. El corrimiento está limitado a valores positivos entre 1 y 8.

15	14	13	11	10	8	7	6	4	3	0
10	rs2			rs1	1	rd			0000	

slli rd, rs1, immediate $x[rd] = x[rs1] \ll \text{immediate}$
Shift Left Logical Immediate. Tipo I.

Corre el registro $x[rs1]$ a la izquierda por immediate posiciones de bits. Los bits liberados son reemplazados por ceros; y el valor es guardado en $x[rd]$. El corrimiento está limitado a valores positivos entre 1 y 8.

15	11	10	8	7	6	4	3	0
immediate			rs1	0	rd	0101		

slt rd, rs1, rs2 $x[rd] = x[rs1] <_s x[rs2]$
Set Less Than. Tipo R.

Compara $x[rs1]$ con $x[rs2]$ como valores signados, y escribe 1 en $x[rd]$, y 0 si no.

15	14	13	11	10	8	7	6	4	3	0
10	rs2			rs1	0	rd			0000	

slti rd, rs1, immediate $x[rd] = x[rs1] <_s \text{sext}(\text{immediate})$
Set Less Than Immediate. Tipo I.

Compara $x[rs1]$ con immediate extendido en signo como valores signados, y escribe 1 en $x[rd]$, y 0 si no.

15	11	10	8	7	6	4	3	0
immediate			rs1	0	rd	0111		

srl rd, rs1, rs2 $x[rd] = x[rs1] \gg x[rs2]$
Shift Right Logical. Tipo R.

Corre el registro $x[rs1]$ a la derecha por $x[rs2]$ posiciones de bits. Los bits liberados son reemplazados por ceros; y el valor es guardado en $x[rd]$. El corrimiento está limitado a valores positivos entre 1 y 8.

15	14	13	11	10	8	7	6	4	3	0
11	rs2			rs1	0	rd			0000	

srl rd, rs1, immediate $x[rd] = x[rs1] \gg \text{immediate}$
Shift Right Logical Immediate. Tipo I.

Corre el registro $x[rs1]$ a la derecha por immediate posiciones de bits. Los bits liberados son reemplazados por ceros; y el valor es guardado en $x[rd]$. El corrimiento está limitado a valores positivos entre 1 y 8.

15	11	10	8	7	6	4	3	0
immediate			rs1	1	rd	0101		

st $rs1, offset$ $M[offset] = x[rs1]$
Store. Tipo S.
 Carga el valor alojado en $x[rs1]$ y lo escribe en la dirección de memoria $offset$.

15	11	10	8	7	6	4	3	0
offset[15:11]	rs1	0	offset[6:4]	0010				

sub $rd, rs1, rs2$ $x[rd] = x[rs1] - x[rs2]$
Subtract. Tipo R.
 Resta el registro $x[rs2]$ al registro $x[rs1]$ y escribe el resultado en $x[rd]$. Overflow aritmético ignorado.

15	14	13	11	10	8	7	6	4	3	0
00	rs2	rs1	1	rd	0000					

subi $rd, rs1, immediate$ $x[rd] = x[rs1] - sext(immediate)$
Subtract Immediate. Tipo I.
 Resta el $immediate$ extendido en signo al registro $x[rs1]$ y escribe el resultado en $x[rd]$. Overflow aritmético ignorado.

15	11	10	8	7	6	4	3	0
immediate[15:11]	rs1	1	rd	0011				

Xor $rd, rs1, rs2$ $x[rd] = x[rs1] \wedge x[rs2]$
Exclusive-OR. Tipo R.
 Calcula el OR exclusivo a nivel de bits entre registro $x[rs1]$ y el registro $x[rs2]$ y escribe el resultado en $x[rd]$.

15	14	13	11	10	8	7	6	4	3	0
01	rs2	rs1	0	rd	0000					

Xori $rd, rs1, immediate$ $x[rd] = x[rs1] \wedge sext(immediate)$
Exclusive-OR Immediate. Tipo I.
 Calcula el OR exclusivo a nivel de bits entre registro $x[rs1]$ e $immediate$ extendido en signo y escribe el resultado en $x[rd]$.

15	11	10	8	7	6	4	3	0
immediate[15:11]	rs1	0	rd	0100				

Capítulo 3

Ensamblador

Se desarrolló un entorno completo para el desarrollo de programas utilizando el lenguaje ensamblador propio de la arquitectura. El entorno (Fig. 3.1) cuenta con 3 paneles: El primer panel posee un espacio en el que se puede editar/escribir el código ensamblador. El segundo panel está dividido de manera vertical: el panel de arriba muestra el proceso de tokenización basado en la gramática formal, mientras que el de abajo muestra las instrucciones ya separadas y catalogadas. El panel de la derecha muestra el programa codificado a binario. Si se llegase a querer, una vez ensamblado un programa, uno puede dar clic en “Explicar Instrucciones” para recibir una explicación corta instrucción por instrucción.

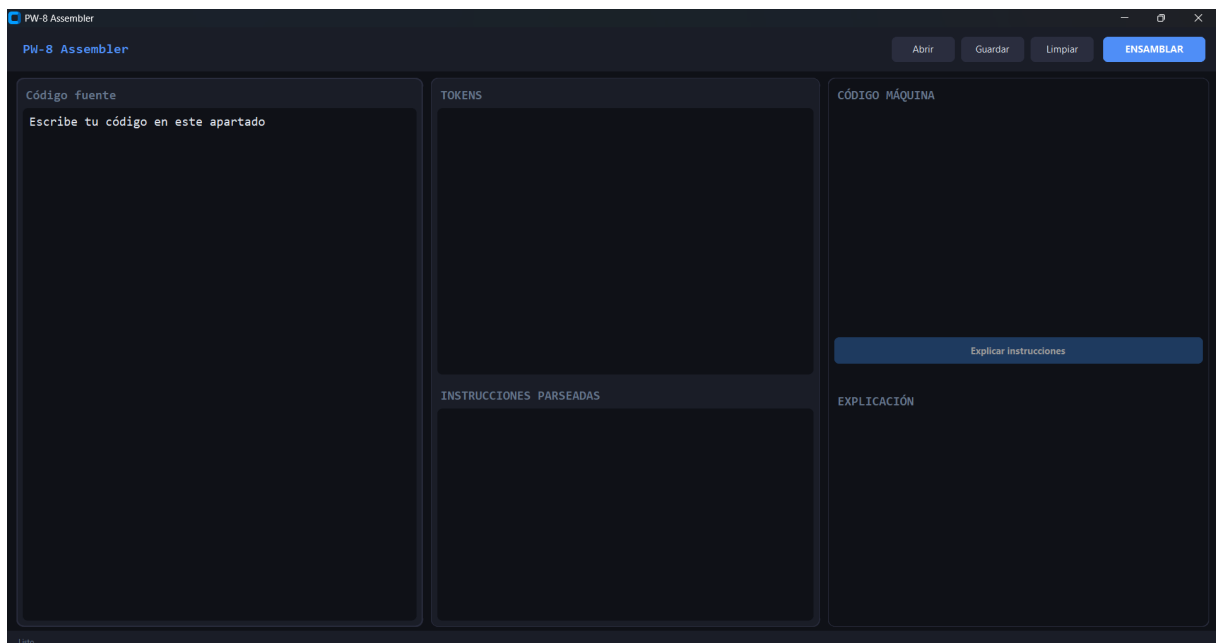


Figura 3.1: UI del ensamblador

3.1. Forma de Uso

El usuario puede escribir el código en la parte izquierda de la UI. En caso de tener el programa escrito en un archivo de texto, éste puede abrirse desde el programa, lo que resulta en la visualización del código en el panel izquierdo. Después, tras dar clic en ‘Ensamblar’, se genera el código en binario, mismo que se muestra en el panel derecho, de manera similar, en los paneles centrales se muestran los resultados del análisis léxico y sintáctico. Si llegase a haber algún error en el código, este será avisado mediante un pop-up en esta etapa.

Después, si se llega a requerir, se puede hacer clic en “Explicar Instrucciones” para visualizar en detalle qué hace cada instrucción. Si todo funciona bien y el programa ensambla correctamente, uno puede hacer clic en “Guardar” que permite guardar el binario en un archivo txt o asm, según se decida.

Capítulo 4

Ejecutar un programa en el procesador

4.1. Cargar Programa

Dentro de los módulos del Procesador está la Memoria de Instrucción (o Memoria de Programa). El contenido está implementado como una ROM. Dicha ROM es donde se incluirá el programa a correr en el procesador. Dentro de la Memoria, encontraremos la siguiente estructura:

```
constant MEM_INSTR : MATRIZ := (  
    -- AQUÍ IRÁ NUESTRO BINARIO  
    others => (others => '0')  
);
```

Una vez ensamblado el programa, copiaremos el binario en la sección indicada. Al ser VHDL, se debe formatear cada binario con comillas y una coma marcando la separación. Por ejemplo, un posible programa sería:

```
constant MEM_INSTR : MATRIZ := (  
    "0000000000010001",  
    "0000000100100001",  
    "0000001000110001",  
    "0000110001000001",  
    "0001000100010000",  
    "0100100100000010",  
    "0001000100100000",  
    "0100101000000010",  
    "0001001100110011",  
    "0010001100100110",  
    "0000010010000110",  
    "0000000000000011",  
    others => (others => '0')  
);
```

4.2. Configuración de salidas

Las salidas marcadas en la FPGA son las correspondientes a los display de 7 segmentos. Está diseñado para funcionar “out-of-the-box” con las tarjetas Nexys 4 de Xilinx. Si se desea cambiar tarjeta, es posible que el módulo “BCDTo7Seg” tenga que ser modificado.

Las salidas DISP_SEL y DISP_VAL son las salidas al display. DISP_VAL está configurado por defecto para mostrar 2 datos: la dirección de memoria en la que se escribe (que empata con la salida del Extensor); y la señal de escritura a la Memoria de Datos (que empata con la salida RD1 del Archivo de Registros). La dirección de memoria del lado izquierdo y el valor del lado derecho.

Con las constantes isVAL1Signed e isVAL2Signed podemos determinar si el valor debe ser considerado signado (rango de -128 a 127) o sin signo (0-255).

Así mismo, existe una señal INS_INDICATOR que se activa cuando se encuentra una instrucción específica. Esto ayuda a identificar en qué momento se ejecutan instrucciones dadas en el programa, lo que permite monitorear de manera sencilla el funcionamiento del programa. La instrucción que se marca por defecto es ST.

Capítulo 5

Ejemplos de Programas

A continuación se mostrarán algunos programas que se podrían desarrollar para con los alumnos, dándoles un entendimiento más profundo de cómo el procesador interpreta y ejecuta distintas instrucciones.

5.1. Contador ascendente iniciando en 7

Este es un contador infinito que comienza en 7 aumenta de 1 en 1 de manera ascendente. El conteo utiliza un segundo registro para el valor (es decir, no utiliza valores inmediatos). Esto permite poder modificar el lugar de inicio y el número de aumento en el conteo (ir de 1 en 1, de 2 en 2, e incluso hacia atrás).

```
LI x1, 7
LI x2, 1

.CICLO
ADD x1, x1, x2

ST x1, 5
JAL x0, CICLO # JAL con x0 no guarda la siguiente dirección, por
               # lo que este es un salto incondicional sin
               # posibilidad de retorno
```

5.2. Secuencia de Fibonacci

Este programa calcula la secuencia de Fibonacci hasta el 89. Lo hace de manera directa con 2 registros que alternan los valores posibles. Incluye la noción de NOP y saltos condicionales que forman bucles.

```
LI x1, 0
LI x2, 1
LI x3, 2 # Cuántos términos hemos obtenido
LI x4, 12

.SIGUIENTE
ADD x1, x1, x2
ST x1, 72 # Guardar en dirección 72
ADD x2, x1, x2
ST x2, 72 # Guardar en dirección 72
ADDI x3, x3, 2

BEQ x3, x4, SALIDA
JAL x0, SIGUIENTE

.SALIDA
ADDI x0, x0, 0 # Equivalente a NOP
```

5.3. Contador con revisor de paridad

A diferencia del contador anterior que únicamente guarda el valor actual, este revisa la paridad de los valores y guarda diferentes valores en memoria dependiendo de si el valor actual es par o impar.

```
LI x1, 0
LI x2, 1

.CHECAPARIDAD
LI x3, 0
ANDI x3, x1, 1
BEQ x3, x2, ESNON

.ESPAR
LI x3, 240 # 0xF0
ST x3, 10 # 0x0A
JAL x0, SIGUE

.ESNON
LI x3, 15 # 0x0F
ST x3, 10 # 0x0A

.SIGUE
ADD x1, x1, x2
JAL x0, CHECAPARIDAD
```

5.4. Obtener el mayor de 3

Este programa recibe 3 valores (es necesario reemplazar VALOR1, VALOR2 y VALOR3 por los valores deseados) y guardará en memoria el mayor de los 3.

```
LI x1, VALOR1
LI x2, VALOR2
LI x3, VALOR3
LI x4, 0
LI x5, 1

SLT x4, x1, x2 # x4 = 1 si x1 < x2
BEQ x4, x5, GR0T1

# x1 >= x2
LI x4, 0
SLT x4, x1, x3 # x4 = 1 si x1 < x3
BEQ x4, x5, GRS3

# VALOR1 es mayor
ST x1, 32 # 0x20
JAL x0, SALIDA

.GR0T1
LI x4, 0
SLT x4, x2, x3 # x4 = 1 si x2 < x3
BEQ x4, x5, GRS3

# VALOR2 es mayor
ST x2, 32 # 0x20
JAL x0, SALIDA

.GRS3
ST x3, 32 # 0x20

.SALIDA
ADDI x0, x0, 0
```